Algorithms. mid-term exam

Question	
1	/32
2	/40
3	/28
Total	/100

Prof. Tami Tamir

5.12.25

Student I.D: _____



Exam length: 100 minutes

- One reference page is allowed.
- You can use draft papers, but you need to answer only on the exam booklet, in the provided space and only on the front pages. Be aware that the back side is not scanned.
- You can cite and rely (without providing a proof) on theorems and claims that were proven in class or in the homework.
- In every question or section marked by ♣, you can write "I don't know" and get one quarter of the score (rounded down).

G O O D L U C K!

Question 1 4*(4+4) = 32 pts.

In each of the following questions, circle the bullet corresponding to the correct answer and justify shortly. In each question, 4 points are for a correct choice and 4 points for the justification.

♣ **1.1** Let $G=(V_1,V_2,E)$ be a complete undirected bipartite, that is, $E=V_1\times V_2$.
It is known that $1< |V_1| < |V_2|$. Let $v_1\in V_1$, $v_2\in V_2$. Let T_1 be the directed spanning tree produced by BFS(v_1). Let T_2 be the directed spanning tree produced by BFS(v_2). Let L_1 be the number of leaves in T_1, and let L_2 be the number of leaves in T_2.
(Recall that a leaf in a directed tree is a node whose out-degree is 0).

- $L_1 < L_2$
- $L_1 = L_2$
- $L_1 > L_2$

Justification:

In both runs, the number of leaves is $|V_1| + |V_2| - 2$. Specifically, all the nodes except from the root and its first neighbor in the other side of the graph are leaves.

♣ **1.2** We run DFS on the De Bruijn graph $G_{2,4}$, which corresponds to the De Bruijn sequence for words of length n=4 over the alphabet Σ={0,1}. We start from the node 011.
The graph is implemented using adjacency lists whose elements are sorted in increasing lexicographic order (for example, in the adjacency list of the node 110, the node 100 appears before 101).

- The node 100 will be labeled before the node 110.
- In the resulting directed spanning tree, the root has two children.
- The resulting spanning tree is a directed path.

Justification:

By definition of $G_{2,4}$, out-degree(011)=2. The first edge is 0110 which leads to 110 will be added to the spanning tree. The second edge is 0111 which leads to to 111. Since the other edge entering 111 is a self-loop, the node 111 will not be marked when the run backtracks to the root 011, implying that the edge 0111 will be added to the spanning tree.

♣ **1.3** Given a directed graph G with arbitrary edge weights and no negative cycles. We run the **Bellman–Ford algorithm** to find shortest paths from a source vertex s. For a vertex v, the field $v.\pi$ (initialized to NULL) is updated during the run **three times**: first to $v.\pi = u$, later to $v.\pi = w$, and later again back to $v.\pi = u$, until the algorithm terminates.

- Such an execution cannot occur**.**
- At least one of the two edges (u,w) or (w,u) is present in the graph.
- The graph includes at least two paths from s to u**.**

Justification:

Given that $v.\pi$ is updated to u twice, we know that the value of $v.d$ is decreased twice when relax(u,v) is performed, that is, when a path that ends with (u,v) is considered. This can only happen if u.d is decreased at least twice – implying that there are at least two different paths from s to u.

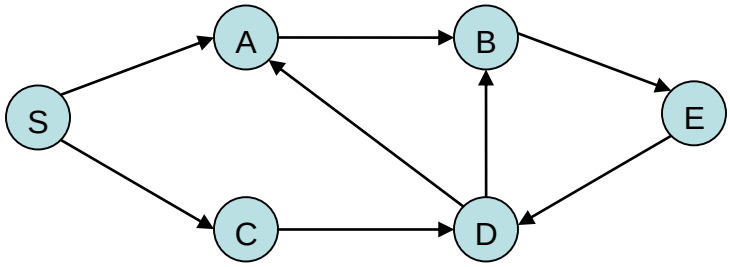
♣ **1.4** Let G be a simple connected undirected graph. It is known that degree(v)=2.
We run **DFS(v)** on the graph and obtain an undirected spanning tree in which degree(v)=1.

- The vertex v participates in at least one cycle.
- The vertex v participates in all cycles in the graph.
- It is possible that the vertex v is not on any cycle, but the graph must contain at least one cycle.

Justification:

Denote by a,b the two neighbors of v in G. Assume w.l.o.g. that a is the first node marked by v in DFS(v). We are given that the edge (v,b) is not in the DFS tree, thus, when the DFS backtracks to v, node b is already marked, implying that there is a path from a to b that does not pass in v. Together with the edges (v,a),(v,b) we get a cycle.

Question 2 (40 points)



♣ **2.1 (6 pts)** Let G be the directed graph shown in the figure. You do *not* need to justify your answers for the following items.

1. Specify an edge such that, if we reverse its direction, the graph G will have an Euler path.

   SA

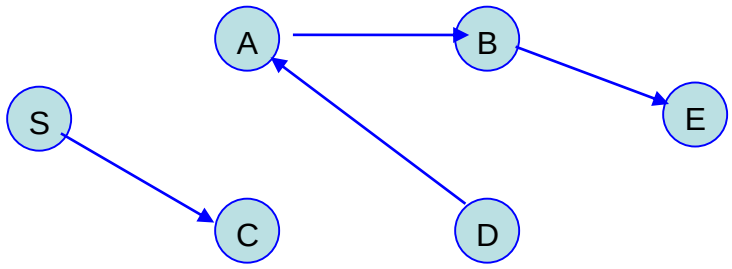
   What is the Euler path in the graph after reversing that edge? ASCDABEDB

2. Specify an edge such that, if we reverse its direction, the graph G will have three different topological sorts. BE

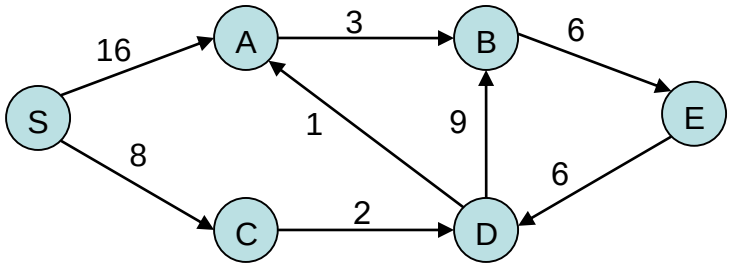
   What are the three topological sorts of the graph after reversing that edge?

   SECDAB, SCEDAB, ESCDABDAB

♣ **2.2 (4 pts)** Draw the directed spanning forest obtained by running **DFS(D)**.
Whenever there is more than one possible vertex to visit next, give priority to vertex **s**, and then to the other vertices in increasing lexicographic order.



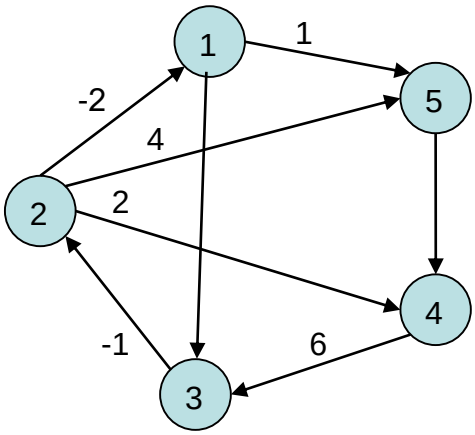
♣ **2.3 (10 pts)** Weights have been added to the edges of the graph as shown in the figure.



Run Dijkstra Algorithm for finding shortest paths from S as source on G. Fill the table. You need to fill the rows according to the order the nodes leave the priority queue. Fill this order in the left column. Note that the last row refers to the parent field.

	S	A	B	C	D	E
v.d init value	0	∞	∞	∞	∞	∞
v.d value after node _S_ leaves the queue	0	16	∞	8	∞	∞
v.d value after node C _ leaves the queue	0	16	∞	8	10	∞
v.d value after node D __ leaves the queue	0	11	19	8	10	∞
v.d value after node _ A _ leaves the queue	0	11	14	8	10	∞
v.d value after node B _ leaves the queue	0	11	14	8	10	20
v.d value after node _ E _ leaves the queue	0	11	14	8	10	20
v.π (parent) at the end	null	D	A	S	C	B

♣ **2.4 (6 pts)** Complete the missing edge-weights so that the resulting weighted graph has no negative cycles, and such that when running the **Floyd–Warshall algorithm** on it, the following holds: $\delta^5(2,3) < \delta^4(2,3) < \delta^1(2,3)$. No need to justify your answer.



w(1,3)= 100

w(5,4)=1

any values such that
-2+1+w(5,4)+6 < 8 < -2+w(1,3)
and no negative cycle is created (w(5,4) should be at least -4, because of the cycle 1-5-4-3-2-1

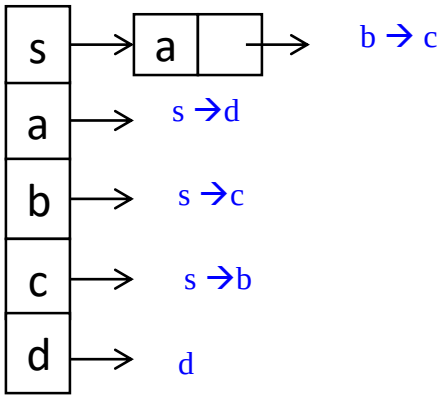
♣ **2.5 (7 pts)** Let G=(V,E) be an **unweighted tournament graph** on n vertices, V={v_1,…,v_n}. It is known that v_1 is a source, that is, in-degree(v_1)=0. Propose a way to assign weights to the edges **and** a way to order the edges, such that if we run the **Bellman–Ford algorithm** on the weighted graph to find shortest paths from v_1, and use **your chosen edge order** in every sweep, then the algorithm requires exactly n−1 sweeps to compute all shortest-path distances, that is, only at the end of the (n-1)st sweep, for all v∈V, v.d=d(v_1,v) holds.
Justify your answer.

Find a Hamiltonian path in the tournament. We proved that such a path exists. Since v1 is a source, v1 must be the first node in this path. Assume w.l.o.g. that the path is v1→v2→⋯→vn.
Assign weight 1 to each edge on the path; assign a large weight W>n to all other edges.
Order the edges in each sweep from right to left along the path: that is, use the order (v_{n-1},v_n),…,(v_1,v_2).

The idea: during each sweep, only the next vertex on the path gets updated.
After n−1 sweeps, v_n.d is computed; thus Bellman–Ford requires exactly n−1 sweeps.

♣ **2.6 (7 pts)** Complete the following adjacency lists so that they represents a **simple, undirected graph** G over V={s,a,b,c,d} with the **minimal number of edges**, such that G has an **Euler path**, and a run of **BFS(s)** on G produces a **tree** whose edges are {(sa),(sb),(sc),(ad)}.

Explain why the graph you describe fulfills the requirements.

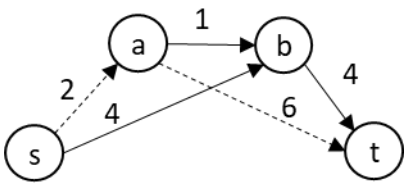


An Euler path is s-b-c-s-a-d.
The number of edges is minimal: The BFS edges must be in. The BFS tree includes four nodes with an odd degree, so at least one additional edge is required, to reduce to 2 the number of nodes with an odd degree.
Note: any edge in {bc,bd,cd} can be added to the BFS tree.

♣ **Question 3 (28 pts.)**

A communication network is given by a directed, weighted graph G=(V,E). All edge weights are positive. Each edge represents a communication line, and the weights correspond to the time required to send a message along that line.
Some communication lines, given by **F⊆E**, are owned by the company *Flower Communication*. You are a customer of the company, and you use the network to send messages from a source node **s** to a target node **t**. The customer retention department offers you the option to upgrade one communication line, which will cut its transmission time in half.
Suggest an efficient algorithm that determines which edge e∈F to upgrade so that the total time to send a message from s to t is minimized.
Describe the algorithm in words and formally, prove its correctness (state the main claim, and justify its correctness in one paragraph), specify and analyze its time complexity.

Example: In the graph shown in the figure, for F={(s,a),(a,t)}, without any upgrade, the fastest path from s to t is s-a-b-t with length 7. If we upgrade the edge (s,a), then the fastest path remains s-a-b-t, and its length decreases to 6. If we upgrade the edge (a,t), then the fastest path becomes s-a-t with length 2+(6/2)=5 which is optimal. Therefore, the algorithm should output (a,t).



Write your answer in the next page.

Question 3 – solution

The important observation is that we are looking for a shortest path from s to t that includes at most one upgraded edge.

Algorithm:
1. Build a graph G'=(V',E'). V'= $V_1 \cup V_2$, each Vi is a copy of V.
E'= {(v1, u1)| for all (v,u)∈E} ∪ {(v2, u2)| for all (v,u)∈E} ∪ {(v1, u2)| for all (v,u)∈F}.
The weight of any edge (vi,ui) is the weight of (v,u).
The weight of any edge (v1,u2) is w(u,v)/2.

2. Run Dijkstra's algorithm on G' with vertex s1 as the source.

3. If d(s1,t1) ≤ d(s1,t2) then no communication link should be upgraded.
If d(s1,t1) > d(s1,t2) then use the father field (v.π) to backtrack from t2, and return the (single) edge from V1 to V2 in the path.

The algorithm builds a graph consisting of two copies of the vertices. Each of the copies includes all the edges with their original costs. In addition, the two copies are connected by edges in F, whose cost is halved. By the construction, every path from s1 to v1 includes no edge with a reduced cost, and every path from s1 to v2 includes exactly one edge from F whose cost is halved.

Claim:
1. There is a path of cost k from s to t in G, if and only if there is a path of length k from s_1 to t_1 in G'.
2. There is a path of cost k from s to t that includes an edge e∈ F in G, if and only if there is a path of length k-w(e)/2 from s_1 to t_2 in G'.
In particular, the shortest path from s_1 to t_2 in G' corresponds to a path that does not make use of the offer, and the shortest path from s_1 to t_2 in G' corresponds to the shortest path from s to t in G that cut by half the cost of one edge from F.

Time complexity:
1. The size of G' is linear in the size of G. Specifically, |V'|=2|V|, |E'|≤2|E|.
Therefore, building G' takes O(|E|+|V|).
2. Running Dijkstra, assuming an implementation with Fibonacci heaps takes O((|E'|+|V'|log|V'|) = O((|E|+|V|log|V|).
3. Considering the two options takes O(1). Backtracking takes O(|V|).
All together, the time complexity is O((|E|+|V|log|V|).

Another possible algorithm:
1. Build a graph G'=(V',E'). $V' = V,\ E' = \{(u, v)|$ for all $(v,u) \in E\}$ (reverse edges)
The weight of any edge (u,v) is the weight of (v,u).
2. Run Dijkstra's algorithm on G with vertex s as the source.
Run Dijkstra's algorithm on G' with vertex t as the source.
4. For every $(v,u) \in F$, compute $val(u,v) = d_G(s,v) + \frac{w(v,u)}{2} + d_{G'}(t,u)$.
5. Let e_F be the edge for which this value is minimal.
If $val(e_F) < d_G(s,t)$ is then upgrade e_F. Otherwise, no communication link should be upgraded.